

AIMS k-mer Exact Retrieval Algorithm

AIMS Research Prototype

1 Scope

AIMS is a k-mer exact-retrieval index for DNA sequence collections. It is not an aligner. Its purpose is to answer the question: “which reference sequences have exact k-mer evidence for this query, and how much did that evidence cost to retrieve?” The system builds a multi- k canonical k-mer index, stores compressed coordinate-aware postings, and answers queries by ordering seed lookups according to estimated information per byte accessed.

This separation is intentional. AIMS performs exact seed generation, exact dictionary lookup, compressed posting retrieval, and candidate accumulation. It does not perform base-level dynamic-programming alignment in the core library. A downstream aligner can consume the candidate list, but benchmark output must label that as a different stage.

2 Design Goals

The implementation is designed around four scientific requirements:

1. exactness of indexed k-mer membership and coordinate reporting,
2. first-class accounting of compressed bytes read and postings decoded,
3. deterministic construction and query output for reproducible experiments,
4. modular stages so benchmarks can compare retrieval against retrieval, not retrieval against alignment.

The current prototype is deliberately k-mer focused. Syncmers, strobemers, learned indexes, and full alignments are outside the active implementation path.

3 Data Model

Each reference sequence is assigned a document identifier d , sequence identifier s , name, and length. In the current command-line tools, one FASTA or FASTQ record is treated as one document and one sequence. This simple mapping makes the baseline easy to validate and can later be generalized to grouped documents.

A posting is a tuple

$$p = (d, s, x, r),$$

where x is the sequence-local coordinate and $r \in \{\text{forward}, \text{reverse}\}$ is the canonical strand.

For a seed key

$$u = (\text{kmer}, k, \text{encoded canonical k-mer}),$$

the index stores:

- document frequency $df(u)$,
- collection frequency $cf(u)$,

- estimated compressed posting bytes,
- a compressed posting block.

Document frequency counts how many documents contain the seed. Collection frequency counts how many coordinate occurrences the seed has. These two values are intentionally separate: a seed can appear many times in one repetitive sequence while having low document frequency, or it can appear once in many documents.

4 Canonical k-mer Encoding

DNA bases are encoded with two bits:

$$A = 0, \quad C = 1, \quad G = 2, \quad T = 3.$$

For every valid k-mer w , AIMS computes the forward encoding $f(w)$ and reverse-complement encoding $rc(w)$. The canonical seed is

$$c(w) = \min(f(w), rc(w)).$$

If $rc(w) < f(w)$, the occurrence is marked as reverse strand. Ambiguous bases split seed extraction runs; k-mers spanning ambiguous bases are not emitted.

4.1 Example

For the k-mer $w = \text{ACG}$, the forward two-bit representation is

$$f(w) = 00\,01\,10_2.$$

The reverse complement is also CGT encoded as 011011₂. The canonical value is the smaller integer. This rule makes a reference occurrence and its reverse-complement query occurrence meet at the same dictionary key while preserving strand as evidence.

5 Index Construction

For each configured $k \in K$, AIMS builds one exact layer:

1. read FASTA or FASTQ records,
2. generate canonical k-mer occurrences,
3. group occurrences by seed key,
4. sort postings by document, sequence, position, and strand,
5. compute df , cf , frequency class, and compressed byte estimates,
6. encode each posting list as a compressed posting block,
7. build a sorted dictionary and an auxiliary hash lookup table.

The index file stores source metadata, sequence metadata, dictionary records, compressed posting blocks, and a checksum footer. The mmap reader keeps compressed posting blocks file-backed and decodes them lazily.

5.1 Construction Pseudocode

```
for each k in requested_k_values:
    postings_by_seed = empty ordered map
    for each reference sequence:
        for each unambiguous k-mer occurrence:
            key = canonical_encode(k-mer)
            postings_by_seed[key].append(document, sequence, position, strand)

for each key in sorted(postings_by_seed):
    sort postings by document, sequence, position, strand
    df = number of unique documents
    cf = number of postings
    encoded_block = delta_varint_encode(postings)
    dictionary.add(key, df, cf, encoded_size, frequency_class)
    posting_store.add(encoded_block)
```

The ordered map and sorted posting lists make construction deterministic. Given the same input records, k values, compiler behavior, and command-line options, the serialized logical contents are stable.

5.2 Dictionary Layer

The exact dictionary is a sorted array of seed keys and metadata, plus an auxiliary hash table from seed key to seed identifier. The sorted array gives a deterministic serialized representation. The hash table gives constant-time lookup during queries. This is a pragmatic baseline: it is not a minimal perfect hash, but it is simple, exact, and easy to validate.

6 Posting Compression

Posting blocks are sorted and encoded with variable-length integers. For adjacent postings, AIMS delta-encodes document identifiers, sequence identifiers within the same document, and positions within the same sequence. Strand is stored as a one-bit logical value encoded as a varint.

This representation is optimized for exact retrieval: compressed bytes read per query are reported as a first-class metric, while postings decoded per query reports the logical posting workload.

6.1 Encoding Rule

For the first posting in a block, document, sequence, and position are stored as absolute values. For later postings:

- document is stored as the difference from the previous document,
- sequence is stored as a difference only when the document is unchanged,
- position is stored as a difference only when the sequence is unchanged,
- strand is stored independently for each posting.

The decoder validates that all bytes are consumed. Extra bytes or impossible strand values are treated as corruption.

7 Query Planning

For query q , AIMS generates canonical k-mers for every indexed k . Repeated query seeds are deduplicated by $(k, seed_id)$ before lookup, but their multiplicity is retained as support weight.

For a seed u , AIMS estimates:

$$w_{idf}(u) = \log \frac{N + 1}{df(u) + 1},$$

where N is the number of reference documents. The information estimate is

$$I(u) = w_{idf}(u) \cdot \text{survival}(u) \cdot \text{length_bonus}(k) \cdot \text{family_reliability}.$$

For the k-mer-only system, family reliability is one. The priority score is

$$P(u) = \frac{I(u)}{\hat{B}(u)},$$

where $\hat{B}(u)$ is the estimated compressed posting byte cost. Query seeds across all k layers are sorted by decreasing $P(u)$.

7.1 Why Information per Byte

Rare seeds usually reduce the candidate space more than frequent seeds. However, rarity alone is not enough: a rare seed with a large posting block or a long decode path may be more expensive than another seed with slightly lower information but much lower byte cost. AIMS therefore ranks by estimated information divided by estimated memory access cost.

This makes the query planner stage-matched to the goal of the prototype. It optimizes seed retrieval and candidate generation, not alignment score.

7.2 Budgeted Queries

The query engine can run without budgets for exact retrieval over all planned seeds. It can also use explicit budgets:

- maximum seed lookups,
- maximum decoded postings,
- maximum candidate records,
- hot-seed thresholds or hot-seed frequency classes.

Budgeted or hot-seed-skipping runs are labeled in metrics. They should not be reported as equivalent to unrestricted exact retrieval unless the benchmark explicitly states the policy.

7.3 Frequency Classes

At build time, every seed is assigned a frequency class from its collection frequency:

$$\text{rare} \rightarrow \text{medium} \rightarrow \text{hot} \rightarrow \text{very_hot}.$$

The thresholds are configurable through the build command. Query-time hot-seed policies can use either a numeric collection-frequency threshold or a stored frequency-class threshold. A class policy such as `--hot-seed-class hot` applies to both hot and very-hot seeds.

8 Candidate Accumulation

AIMS maintains a hash-map accumulator keyed by (d, s, r) . For each decoded posting, the candidate score and support count are updated. The top- k candidates are emitted with document ID, sequence ID, sequence name, strand, supporting seed count, and score.

Hot seeds can be handled in two modes:

skip omit seeds above the configured collection-frequency threshold or frequency class;

doc-only decode the hot posting block but count at most one contribution per (d, s, r) .

8.1 Scoring Interpretation

The candidate score is a retrieval score, not an alignment score. It measures accumulated exact-seed evidence after planner weighting. Two candidates with similar retrieval scores may still align differently if a downstream aligner is used. Therefore, AIMS output should be described as candidate retrieval or exact seed retrieval.

9 Serialization and Loading

The serialized index contains a magic value, version, endian marker, source metadata, sequence metadata, layer directories, dictionaries, posting blocks, and a checksum. The reader validates the format before exposing the index to query code.

Two loading modes are supported:

- copy mode, where posting blocks are read into process memory,
- mmap mode, where posting blocks remain file-backed and are decoded lazily.

The mmap mode is important for memory experiments because compressed posting bytes can remain outside eagerly allocated heap storage. Benchmarks must report the loading mode because memory and latency behavior differ.

10 Budgets and Metrics

The query engine supports budgets for maximum seeds, maximum decoded postings, maximum candidates, hot-seed thresholds, and hot-seed frequency classes. Budgeted or skipped-seed runs are labeled as heuristic in benchmark output.

Every query reports:

- exact bytes read,
- postings decoded,
- seeds generated and queried,
- skipped hot and budgeted seeds,
- skipped frequency-class seeds and doc-only hot seeds,
- candidate counts and peak accumulator memory,
- per- k metrics,
- structured top- k results,
- elapsed nanoseconds.

10.1 Primary Metric Definitions

Metric	Meaning
<code>exact_bytes_read</code>	compressed posting bytes touched by exact lookup
<code>postings_decoded</code>	logical coordinate postings decoded
<code>seeds_generated</code>	query k-mers generated before dictionary filtering
<code>seeds_queried</code>	unique dictionary seeds whose postings were fetched
<code>seeds_skipped_frequency_class</code>	query seeds skipped by class-aware hot policy
<code>seeds_doc_only_hot</code>	hot seeds counted once per candidate rather than per co-ordinate
<code>candidates_created</code>	distinct document/sequence/strand accumulator entries
<code>elapsed_ns</code>	wall-clock query time in nanoseconds

These metrics are emitted through TSV and JSONL so experiments can be reproduced and inspected with independent scripts.

11 Benchmarking

Benchmarks are stage-labeled as `exact_retrieval`. Truth-based top-1, top-5, and top-10 recall are computed when a truth manifest is supplied. Loading mode (`copy` or `mmap`), cache size, query budgets, and hot-seed policy are part of the benchmark configuration and must be reported with results.

12 Limitations

The current AIMS prototype does not claim to replace an aligner. It does not compute edit distance, CIGAR strings, base-level mapping qualities, splice-aware alignments, or chaining-based final placements. It is a benchmarkable exact k-mer retrieval engine. Its value is in measuring how seed ordering, compressed posting access, and memory-aware lookup affect candidate generation.

13 Expected Use

A typical experiment builds an index once, validates it, queries a FASTA or FASTQ workload, and records per-query metrics:

```
aims_build --ref refs.fa --out idx.aims --k 15,19,23,27,31
aims_validate --index idx.aims
aims_query --index idx.aims --query queries.fa --topk 10 --emit jsonl
aims_bench --index idx.aims --query queries.fa --truth truth.tsv
```

The resulting output should be interpreted as exact seed retrieval. Any downstream alignment handoff should be benchmarked and labeled separately.